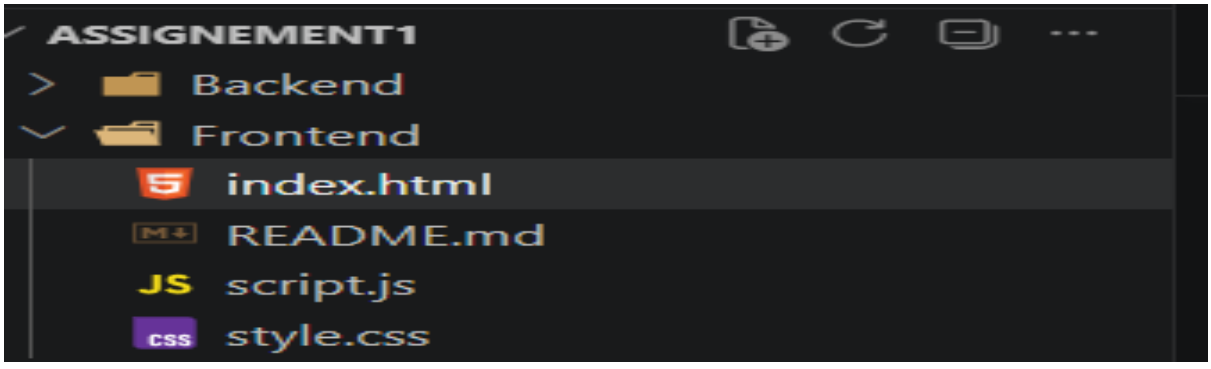
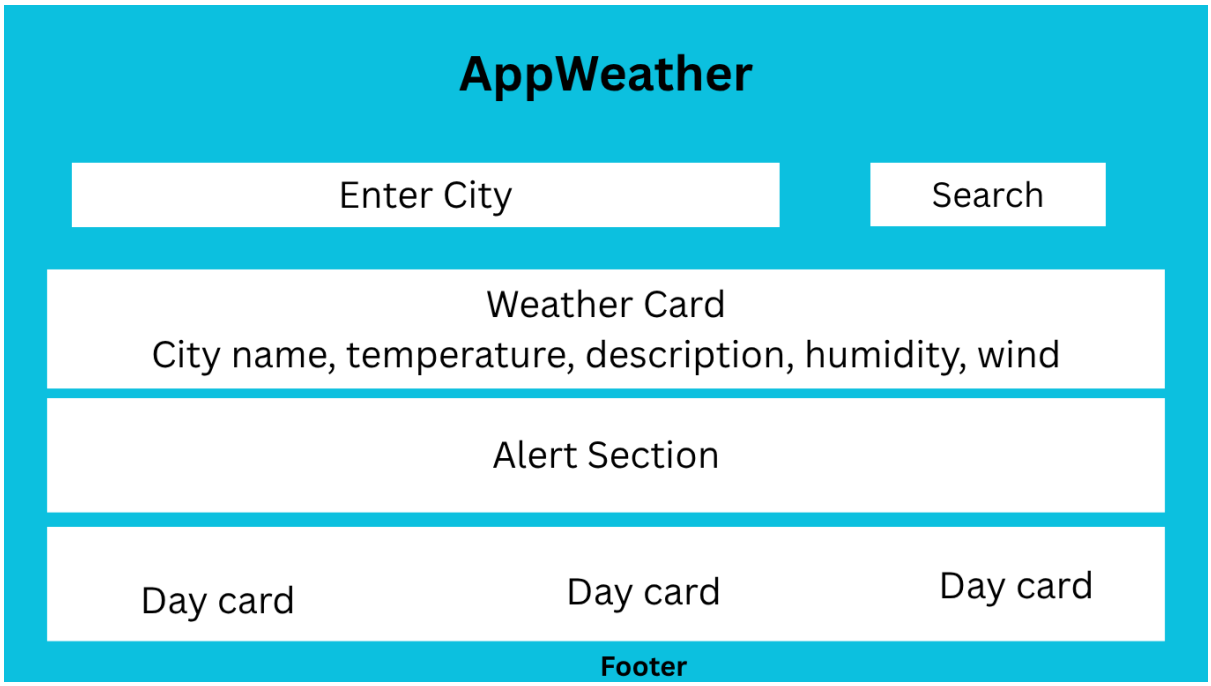
Assignment 1:

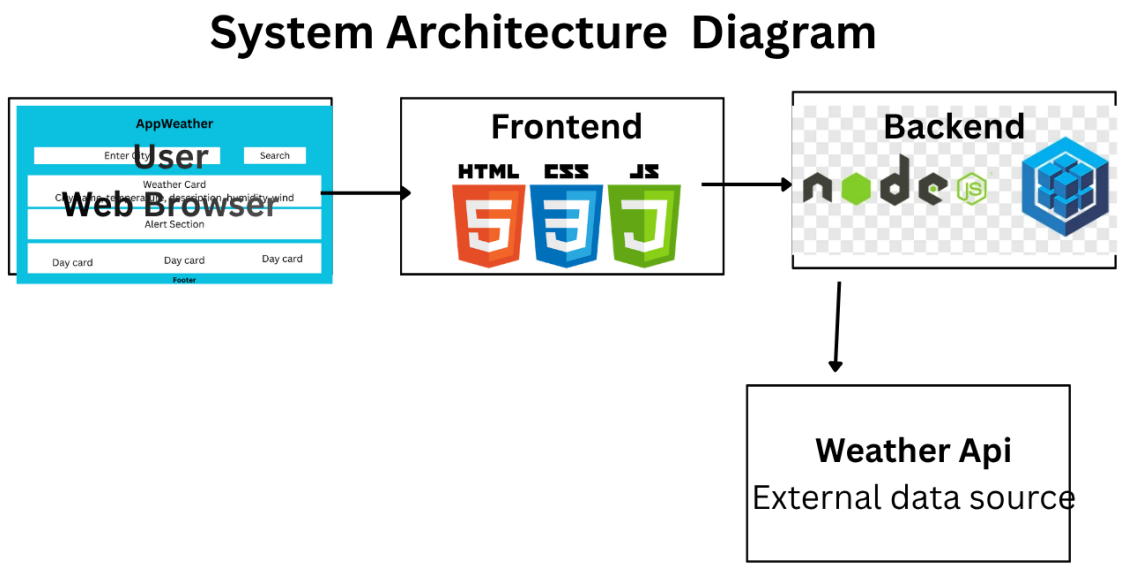
```
 Project Scope & Functional Requirements Document

## Project Scope:
The Weather Forecast & Alert System is a web application that allows users to
search for the current weather and upcoming forecast for any city, receiving
automatic alerts when dangerous weather conditions are detected (extreme heat,
strong winds, storms).

## Objectives:
Provide accurate, real-time weather information
Alert users about risky weather conditions
Offer a simple, responsive, and accessible interface

##  Functional Requirements:
The user must be able to search for weather by city name
The system must display temperature, humidity, wind speed, and weather
description
The system must show a forecast for the next 3-5 days
The system must generate automatic alerts based on predefined conditions
(e.g., temp ≥ 35°C, wind ≥ 60km/h)
The system must be responsive (works on both mobile and desktop)
(Future) The system must support user authentication and email notifications

##
```

1. **HTML,CSS & JavaScript File**



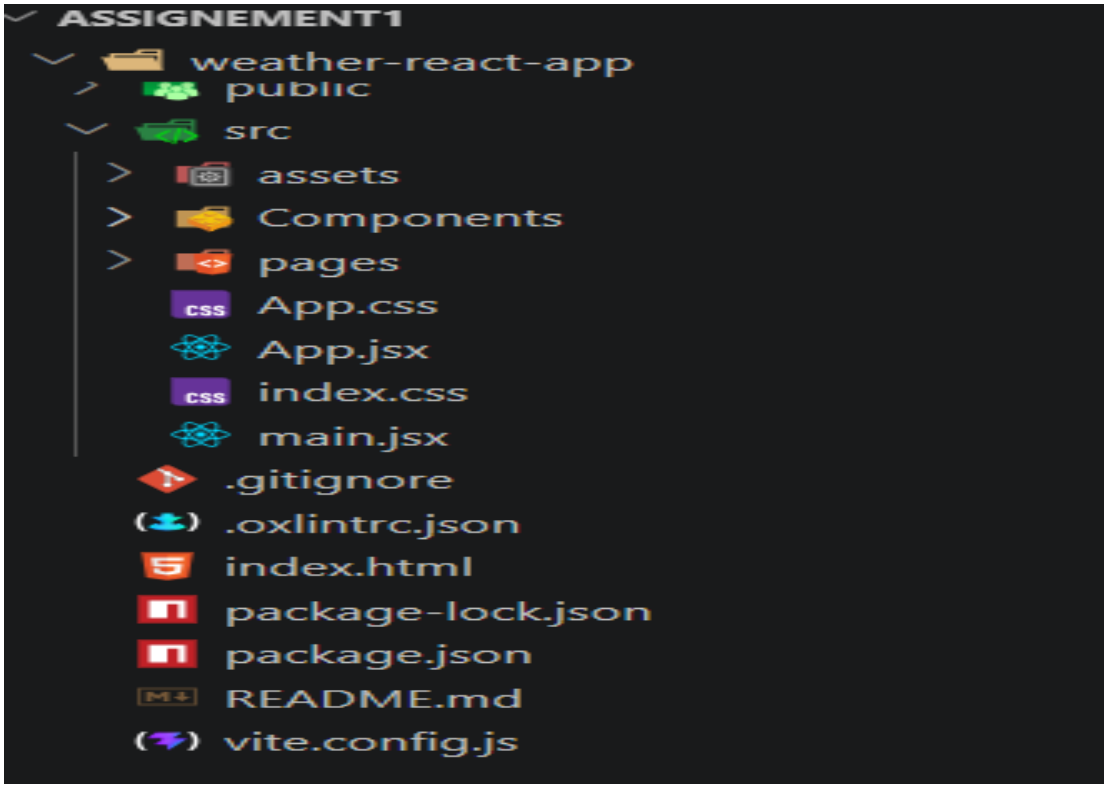
UI designer:


AppWeather
Enter City
Search
Weather Card
City name, temperature, description, humidity, wind
Alert Section
Day card
Day card
Day card
Footer

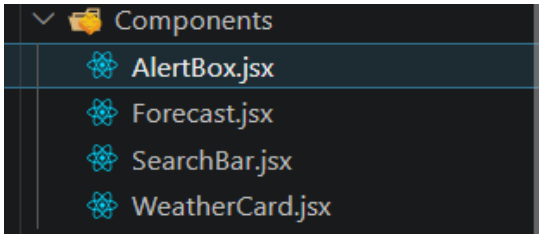

System Architecture Diagram
User
Web Browser
Frontend
HTML
CSS
JS
Backend
Weather Api
External data source

Assignment 2:

React Project Files:



Source Code:



- AlertBox.jsx

```jsx
function AlertBox({ data }) {
  if (!data) return null

  let message = null

  if (data.temp >= 35) {
    message = '⚠ Extreme heat alert!'
  } else if (data.windSpeed >= 60) {
    message = '⚠ Strong wind alert !'
  }

```

```
    if (!message) return null

    return <div className="alert-box">{message}</div>
  }

  export default AlertBox
```

Forecast Code:

```
function Forecast({ forecast }) {
 if (!forecast || forecast.length === 0) return null

 return (
  <div className="forecast-section">
   {forecast.map((item, index) => (
    <div className="forecast-card" key={index}>
     <p>{item.day}</p>
     <strong>{item.temp}°C</strong>
    </div>
   ))}
  </div>
 )
}

export default Forecast
```

SearchBar.jsx

```
function Forecast({ forecast }) {
 if (!forecast || forecast.length === 0) return null

 return (
  <div className="forecast-section">
```

```
      {forecast.map((item, index) => (
        <div className="forecast-card" key={index}>
          <p>{item.day}</p>
          <strong>{item.temp}°C</strong>
        </div>
      ))}
    </div>
  )
}

export default Forecast
```

WeatherCard.jsx:

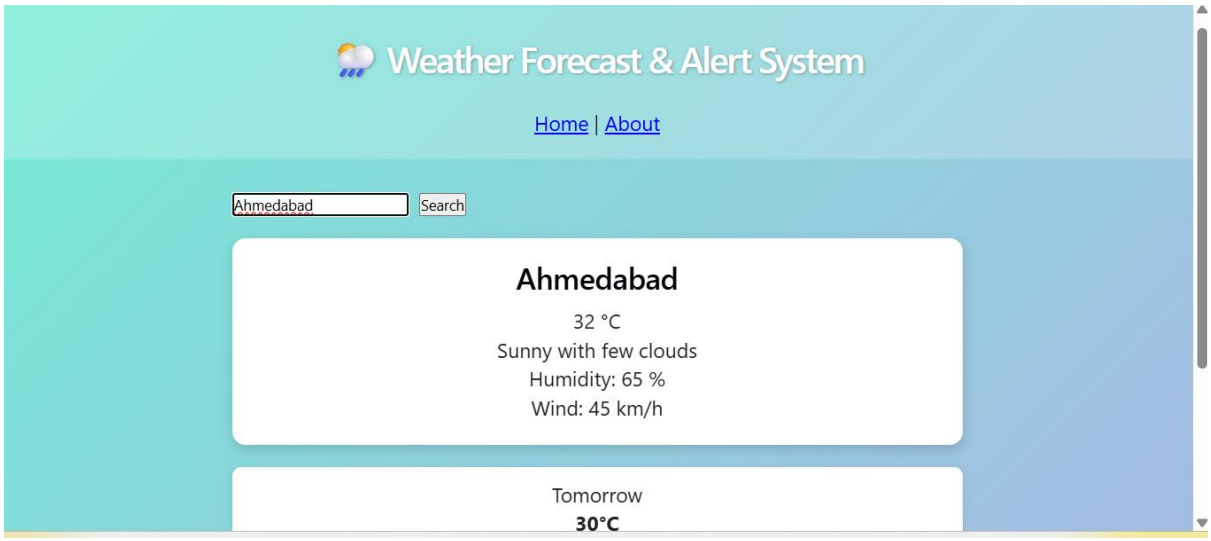
```
function WeatherCard({ data }) {
  if (!data) return null

  return (
    <div className="weather-card">
      <h2>{data.city}</h2>
      <p className="temperature">{data.temp} °C</p>
      <p>{data.description}</p>
      <p>Humidity: {data.humidity} %</p>
      <p>Wind: {data.windSpeed} km/h</p>
    </div>
  )
}

export default WeatherCard
```

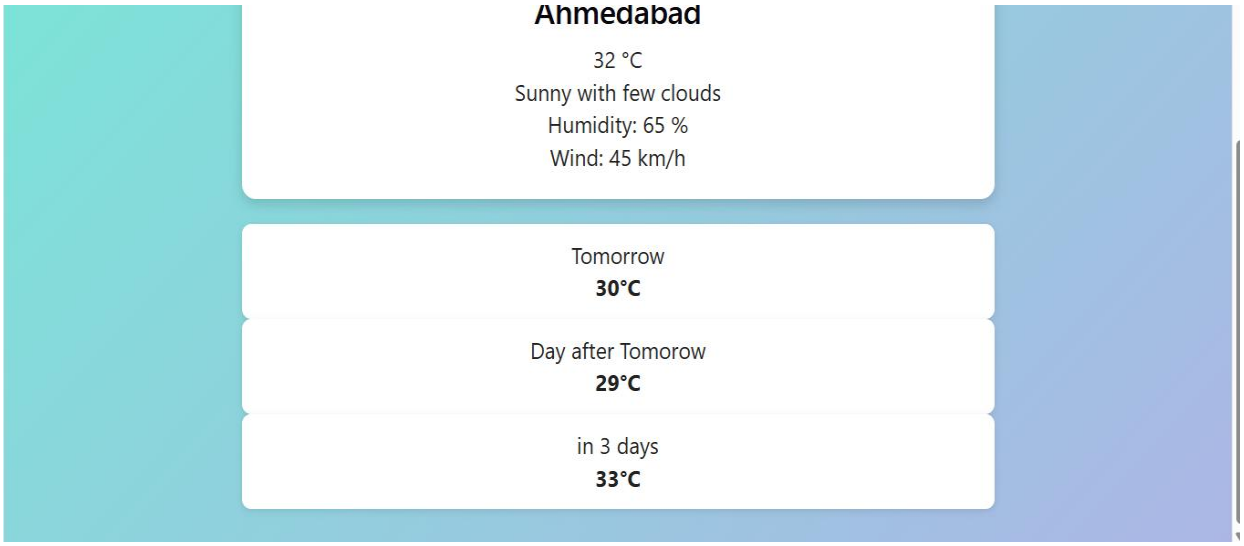
Pages/Homes:

```
import { useState } from 'react'
import SearchBar from '../components/SearchBar'
import WeatherCard from '../components/WeatherCard'
import AlertBox from '../components/AlertBox'
import Forecast from '../components/Forecast'

const mockWeatherData = {
  temp: 32,
  description: 'Sunny with few clouds',
  humidity: 65,
  windSpeed: 45,
  forecast: [
    { day: 'Tomorrow', temp: 30 },
    { day: 'Day after Tomorow', temp: 29 },
    { day: 'in 3 days', temp: 33 }
  ]
}

function Home() {
  const [weatherData, setWeatherData] = useState(null)

  function handleSearch(city) {
    setWeatherData({ ...mockWeatherData, city })
  }

  return (
    <main className="container">
      <SearchBar onSearch={handleSearch} />
      <WeatherCard data={weatherData} />
      <AlertBox data={weatherData} />
```

```
    <Forecast forecast={weatherData?.forecast} />
   </main>
  )
}
```

Screenshots of User Interface:





Frontend Architecture Summary — Weather Forecast & Alert System

The frontend was rebuilt using React.js, adopting a component-based architecture to replace the static HTML/CSS/JS version from the previous assignment. The application is structured into small, reusable components (**SearchBar, WeatherCard, AlertBox**, Forecast), each responsible for a single piece of the UI. This follows the React principle of separation of concerns, making the code easier to maintain and extend.

State management is handled using React Hooks, specifically **useState**. The weather data is stored as state inside the Home page component and passed down to child components via props — a pattern known as "lifting state up." When the user searches for a city, the **SearchBar** component triggers a callback function that updates the shared state, causing React to automatically re-render all components that depend on that data.

Navigation between pages was implemented using React Router (react-router-dom). The application currently has two routes: / (Home, the weather dashboard) and /about (project information). Client-side routing was used, allowing page transitions without full browser reloads, which improves performance and user experience.

Currently, weather data is simulated using mock data, since API integration is planned for the next phase of the project. The architecture is designed so that replacing the mock data with real API calls (using Axios or Fetch) will require minimal changes — only the data-fetching logic inside **Home.jsx** needs to be updated, while all components remain unchanged.